



Projet
Interfaçage d'une diode laser par
microcontrôleur

Auteurs :
Arthur ALLAEYS
Elisa CHARRON

Professeur :
Antoine COURTAY

13 mai 2026

Introduction

Le but de ce projet est de réaliser le contrôle en courant d'une diode laser. Pour cela, le système sera composé d'une carte microcontrôleur ATxMEGA128A1 chargée de contrôler la diode laser. Le système devra pouvoir permettre de choisir le courant traversant la diode et de le mesurer en retour. La carte microcontrôleur sera connectée à un PC sur lequel une interface en ligne de commande (Terminal Putty) permettra d'envoyer des ordres au système. La figure 1, montre comment les différents éléments communiquent entre eux.



FIGURE 1 – Schéma de la communication entre les éléments.

La carte diode laser est représentée sur la figure 2. Elle est reliée au microcontrôleur par une interface I2C (SDA / SCL) et doit également être alimentée (+5V / GND). L'interface I2C permet au microcontrôleur de contrôler 2 composants : un DAC et un ADC, comme schématisé sur la figure 3. La partie analogique du montage permet de gérer le courant traversant la diode comme expliqué dans la section étude du montage.

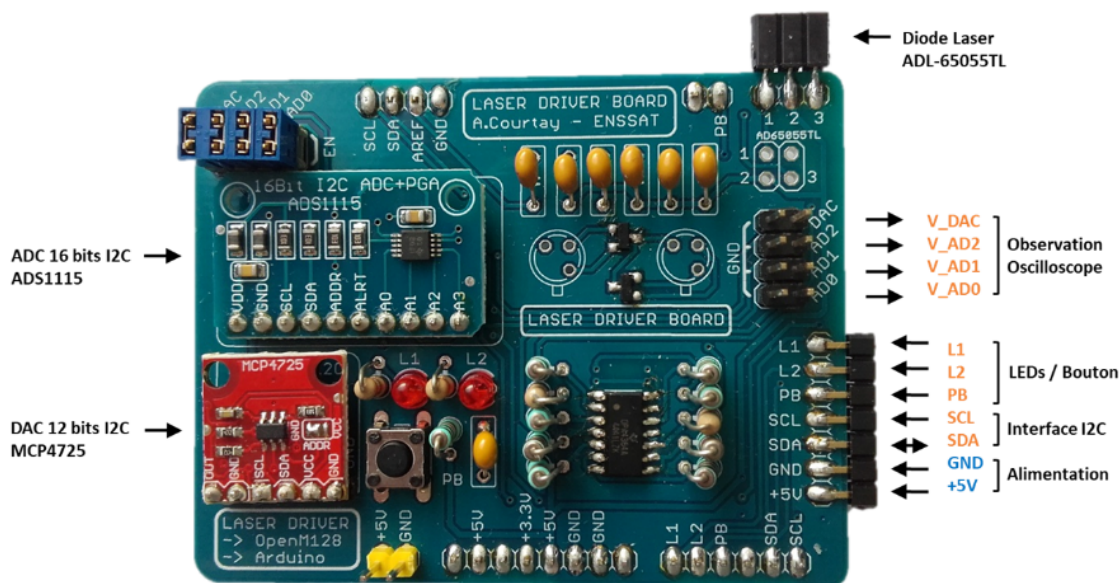


FIGURE 2 – Représentation de la carte diode laser utilisée.

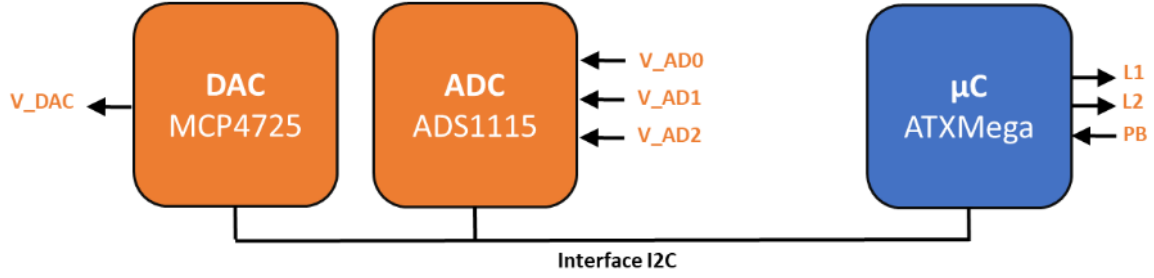


FIGURE 3 – Schéma de l'interface I2C.

1 Etude du montage : carte diode laser

Afin d'asservir le courant traversant la diode laser, il est important de pouvoir en retour mesurer ce courant que nous venons de positionner par le DAC. Pour cela il est nécessaire de lire certaines tensions sur le montage à l'aide d'un ADC (*Analog to Digital Converter*) commandé également en I2C par le microcontrôleur. Nous allons donc chercher à établir l'expression de ces tensions, en nous appuyant sur le schéma électrique de la carte, figure 4.

Calcul de I_POLAR_TH , le courant de polarisation théorique.

On considère que $I_E = I_POLAR_TH$, donc $I_B = 0$. Ce sera valable pour toute l'étude théorique.

De plus, I_POLAR_TH est constant donc tous les courants sont constants, et $I_{6b} = C \frac{du_C}{dt} = 0$.

On en déduit que $I_6 = I_{6a} = I_5$. On applique une première loi des mailles (dans la maille contenant les résistances R_6 , R_7 et le transistor T2) :

$$I_6 R_6 = V_{BE} + I_POLAR_TH \times R_7$$

On fait une deuxième loi des mailles (dans la maille contenant les résistances R_5 et R_6) :

$$V_{cc} = I_6 (R_5 + R_6)$$

On déduit des deux équations précédentes l'équation donnant I_POLAR_TH :

$$I_POLAR_TH = \frac{V_{cc} R_6 / (R_5 + R_6) - V_{BE}}{R_7}$$

On réalise l'application numérique avec $V_{cc} = 5\text{ V}$, $R_5 = 1.2\text{ k}\Omega$, $V_{BE} = 0.6\text{ V}$, $R_6 = 1.5\text{ k}\Omega$, et $R_7 = 120\text{ }\Omega$:

$$I_POLAR_TH = 18.15\text{ mA}$$

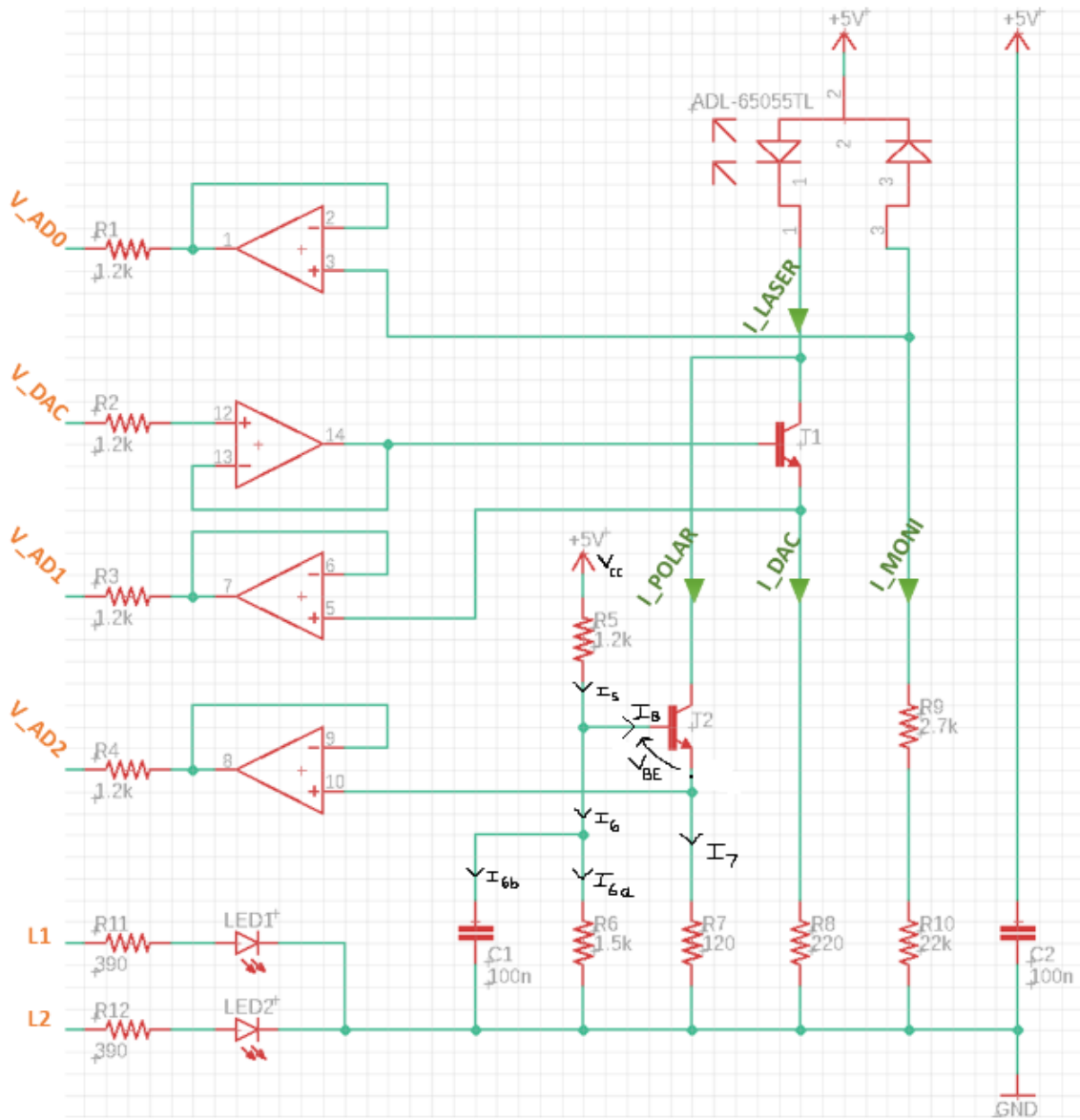


FIGURE 4 – Schéma électrique de la carte diode laser.

Calcul de I_POLAR_READ , le courant de polarisation expérimental issu de la lecture de V_AD2 .

L'AOP est en montage suiveur, donc les tensions au niveau des broches 10 (non inverseuse) et 8 (sortie) sont égales, de valeur V_AD2 .

On fait une loi des mailles dans la maille contenant R_7 :

$$V_AD2 = I_POLAR_READ \times R_7$$

On en déduit l'équation donnant I_POLAR_READ :

$$I_POLAR_READ = V_AD2 / R_7$$

En réalité, quand on utilise la carte, la valeur V_AD2 reçu par l'ADC est un entier $AD2_Value$ compris entre 0 (correspondant à une tension nulle) et $ADC_Full = 2^{16}$ (correspondant à la tension maximale de 5 V).

Finalement :

$$I_POLAR_READ = \left(AD2_Value \times \frac{5}{2^{16}} \right) / R_7$$

Calcul de I_MONI_READ , le courant de monitoring expérimental issu de la lecture de V_AD0 .

L'AOP est en montage suiveur donc les tensions au niveau des broches 3 (non inverseuse) et 1 (sortie) sont égales, de valeur V_AD0 .

On fait une loi des mailles dans la maille contenant R_9 et R_{10} :

$$V_AD0 = I_MONI_READ \times (R_9 + R_{10})$$

On en déduit l'équation donnant I_MONI_READ :

$$I_MONI_READ = V_AD0 / (R_9 + R_{10})$$

Donc avec la valeur numérique $AD0_Value$ de V_AD0 relevée par la carte :

$$I_MONI_READ = \left(AD0_Value \times \frac{5}{2^{16}} \right) / (R_9 + R_{10})$$

Calcul de I_DAC_READ , le courant DAC expérimental issu de la lecture de V_AD1 .

L'AOP est en montage suiveur donc les tensions au niveau des broches 5 (non inverseuse) et 7 (sortie) sont égales, de valeur V_AD1 .

On fait une loi des mailles dans la maille contenant R_8 :

$$V_AD1 = I_DAC_READ \times R_8$$

On en déduit l'équation donnant I_DAC_READ :

$$I_DAC_READ = V_AD1/R_8$$

Donc avec la valeur numérique $AD1_Value$ de V_AD1 relevée par la carte :

$$I_DAC_READ = \left(AD1_Value \times \frac{5}{2^{16}} \right) / R_8$$

Calcul de DAC_Value , la valeur numérique à positionner sur le DAC (V_DAC) pour avoir la valeur voulue I_LASER_WRITE .

L'AOP est en montage suiveur donc les tensions au niveau des broches 12 (non inverseuse) et 14 (sortie) sont égales, de valeur V_DAC .

D'après la loi des noeuds :

$$I_LASER_WRITE = I_POLAR_READ + I_DAC$$

On fait une loi des mailles dans la maille contenant R_8 et le transistor T1 :

$$V_DAC = I_DAC \times R_8 + V_{BE}$$

On déduit des deux équations suivantes l'équation donnant V_DAC :

$$V_DAC = (I_LASER_WRITE - I_POLAR_READ) \times R_8 + V_{BE}$$

La valeur DAC_Value à envoyer au DAC est un entier compris entre 0 (correspondant à une tension nulle) et 2^{12} (correspondant à la tension maximale de 5 V).

On en déduit que :

$$DAC_Value = V_DAC \times \frac{2^{12}}{5}$$

Finalement :

$$DAC_Value = ((I_LASER_WRITE - I_POLAR_READ) \times R_8 + V_{BE}) \frac{2^{12}}{5}$$

Calcul de I_LASER_READ , le courant expérimental traversant la diode laser.

D'après la loi des noeuds :

$$I_LASER_READ = I_POLAR_READ + I_DAC_READ$$

Avec les deux équations de I_POLAR_READ et I_DAC_READ , on obtient l'expression de I_LASER_READ suivante :

$$I_LASER_READ = (AD2_Value/R_7 + AD1_Value/R_8) \frac{5}{2^{16}}$$

2 Pilotage de la carte diode laser

Nous allons maintenant devoir définir un ensemble de commandes permettant de piloter le système. Ces commandes seront saisies dans un terminal sur un ordinateur, sous forme de chaînes de caractères. Ce terminal permet d'envoyer ces commandes sur une interface série UART vers le microcontrôleur. Il faudra donc être en mesure de les récupérer et de les décoder. Les affichages dans le terminal se feront en utilisant les paramètres de transmission par défaut de l'interface UART (9600 bauds / 8 data bits / 1 stop bit / No parity).

Question 1 : Pilotage des LEDs/BP/Buzzer

Nous cherchons à réaliser un programme permettant l'allumage alterné d'une des 2 LED, ainsi que la génération de 2 notes sur appui du bouton poussoir.

Initialement, le code est comme ceci :

```
1 void BUZZ_Go(uint16_t Freq)
2 {
3     while(1){
4         PORTQ.OUT |= (1<<2);
5         _delay_us(10000000/(2*Freq));
6         PORTQ.OUT &= ~(1<<2);
7         _delay_us(10000000/(2*Freq));
8     }
9 }
```

Néanmoins, il s'avère que `delay_us` n'accepte en paramètre qu'une valeur constante, or `Freq` est considérée comme variable. Pour palier cela, nous incluons une structure `switch`, afin de pouvoir écrire directement, en paramètre `delay_us`, la valeur correspondante à la fréquence voulue. Nous avons toujours un problème : le buzzer ne s'arrête jamais. De ce fait, nous enlevons la boucle `while`, et obtenons la fonction `BUZZ_Go`.

```
1 void BUZZ_Go(uint16_t Freq)
2 {
3     switch(Freq){
4         case 0 :
5             break;
6         case 440 :
7             PORTQ.OUT |= (1<<2);
8             _delay_us(10000000/(2*440));
9             PORTQ.OUT &= ~(1<<2);
10            _delay_us(10000000/(2*440));
11            break;
12        case 880 :
13            PORTQ.OUT |= (1<<2);
14            _delay_us(10000000/(2*880));
15            PORTQ.OUT &= ~(1<<2);
16            _delay_us(10000000/(2*880));
17            break;
18        default :
```

```

19     break;
20 }
21 }

```

Concernant la partie `main`, on utilise une boucle `while`, qui ne se trouve plus dans la fonction `BUZZ_Go`. De plus, pour éviter que le buzzer ne se coupe, nous le faisons jouer tout le temps (ligne 13), sauf quand il doit changer de note (i.e. de fréquence).

```

1  int main(void)
2  {
3      _32MHz_Clock_Init();
4      twiInitialize(TWI);
5
6      uint16_t Freq = 0;
7      uint8_t State = 0;
8      BP_Init();
9      LED_Init();
10     BUZZ_Init();
11     while(1)
12     {
13         BUZZ_Go(Freq);
14         if(BP_Read()){
15             switch(State){
16                 case 0 :
17                     LED_Off(1);
18                     LED_Off(2);
19                     Freq = 0;
20                     //BUZZ_Go(0);
21                     State ++;
22                     break;
23                 case 1 :
24                     LED_On(1);
25                     LED_Off(2);
26                     Freq = 440;
27                     //BUZZ_Go(440);
28                     State ++;
29                     break;
30                 case 2 :
31                     LED_On(2);
32                     LED_Off(1);
33                     Freq = 880;
34                     //BUZZ_Go(880);
35                     State = 0;
36                     break;
37                 default :
38                     LED_Off(1);
39                     LED_Off(2);
40                     Freq = 0;
41                     //BUZZ_Go(0);
42                     State = 0;
43                     break;
44             }
45         }
46     }
47 }

```


Question 2 : Pilotage de la liaison série

Réalisons le programme permettant la lecture (avec écho) d'une chaîne de caractère sur la liaison série.

Note : Pour lancer ce programme, nous devons d'abord trouver, à l'aide du gestionnaire de périphériques, sur quel port COM la carte est connectée. Ensuite, nous devons ouvrir PuTTY, sélectionner *Serial*, puis indiquer le numéro du port.

Pour réaliser ce programme, nous devons lire et récupérer, avec `USART0_Read_String`, la chaîne de caractères écrite par l'utilisateur. Ici, nous stockons la chaîne récupérée dans la variable `chaine`. Puis, nous utilisons la fonction `USART0_Write_String` pour afficher cette chaîne. La figure 5 montre le résultat de ce programme, lorsque l'on entre la chaîne "interfacage".

```
1 int main(void)
2 {
3     _32MHz_Clock_Init();
4     twiInitialize(TWI);
5
6     USART0_Init();
7     while(1)
8     {
9         char* chaine;
10        USART0_Read_String(chaine);
11        USART0_Write_String(chaine);
12    }
13 }
```



FIGURE 5 – Ecran obtenu sur PuTTY lors du test de la fonction écho.

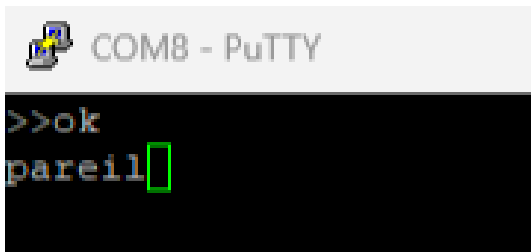
Réalisons maintenant le programme permettant la comparaison de la chaîne reçue avec une autre chaîne. Pour ce faire, nous réutilisons les fonctions `USART0_Read_String` et `USART0_Write_String`. Nous utilisons également la fonction `strcmp`, afin de comparer les chaînes de caractères. La figure 6 montre le résultat de ce programme. Nous en concluons que nous arrivons bien à comparer des chaînes de caractères.

```
1 int main(void)
2 {
3     _32MHz_Clock_Init();
4     twiInitialize(TWI);
5
6     USART0_Init();
```

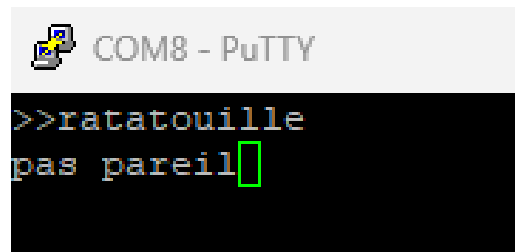
```

7  char chaine1[3] = "ok";
8  char *chaine2;
9  char ok[7] = "pareil";
10 char not_ok[11] = "pas pareil";
11 USART0_Read_String(chaine2);
12 if (!strcmp(chaine1, chaine2))
13 {
14     USART0_Write_String(ok);
15 }
16 else{
17     USART0_Write_String(not_ok);
18 }
19 }

```



(a) Comparaison de "ok" avec "ok".



(b) Comparaison de "ok" avec "ratatouille".

FIGURE 6 – Ecran obtenu sur PuTTY lors de la comparaison de deux chaînes de caractères avec la chaîne "ok".

2.1 Question 3 : Pilotage de l'ADC

Le programme suivant permet de lire les valeurs de tension des entrées de l'ADC : V_{AD0} , V_{AD1} et V_{AD2} . Se sont des valeurs numériques comprises entre 0 et 2^{16} , qui permettent de faire une conversion pour remonter par la suite aux valeurs de tension qui sont, elles, comprises entre 0 et 5V (2^{16} correspondant à 5V).

Dans ce programme, on utilise la fonction `ADS115_Read` qui permet de lire la valeur de l'entrée de l'ADC souhaitée. Il écrit ensuite cette valeur dans le terminal PuTTY avec la fonction `USART0_Write_String`. Nous obtenons alors l'affichage de la figure 7.

```

1  int main(void)
2  {
3      char Full_Command[100];
4      char Name_Command[6];
5
6      _32MHz_Clock_Init();
7      twiInitialize(TWI);
8
9      USART0_Init();
10
11     char *pV_AD0 = NULL;
12     char *pV_AD1 = NULL;
13     char *pV_AD2 = NULL;
14

```

```

15 unsigned long AD0_Value = ADS1115_Read(0);
16 unsigned long AD1_Value = ADS1115_Read(1);
17 unsigned long AD2_Value = ADS1115_Read(2);
18
19 sprintf(pV_AD0, "V_AD0 = %lu\n\r", AD0_Value);
20 USART0_Write_String(pV_AD0);
21 sprintf(pV_AD1, "V_AD1 = %lu\n\r", AD1_Value);
22 USART0_Write_String(pV_AD1);
23 sprintf(pV_AD2, "V_AD2 = %lu\n\r", AD2_Value);
24 USART0_Write_String(pV_AD2);
25 }

```

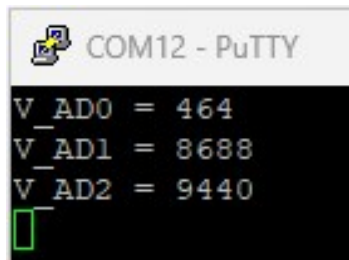


FIGURE 7 – Affichage du terminal PuTTY pour la lecture des valeurs numériques V_AD0, V_AD1 et V_AD2.

Le programme précédent nous a servi de base pour écrire les fonctions IMON_ASK, IDAC_ASK, IPOL_ASK et ILAS_ASK, qui se trouvent en annexe. Elles lisent les valeurs numériques des entrées de l'ADC, et calculent les courants expérimentaux grâce aux expressions de la partie 1 sur l'étude du montage de la carte. Ces valeurs de courant sont ensuite affichées dans le terminal PuTTY comme sur la figure 8.

```

1 int main(void)
2 {
3     char Full_Command[100];
4     char Name_Command[6];
5
6     _32MHz_Clock_Init();
7     twiInitialize(TWI);
8
9     USART0_Init();
10
11     ILAS_ASK();
12     IPOL_ASK();
13     IMON_ASK();
14     IDAC_ASK();
15 }

```

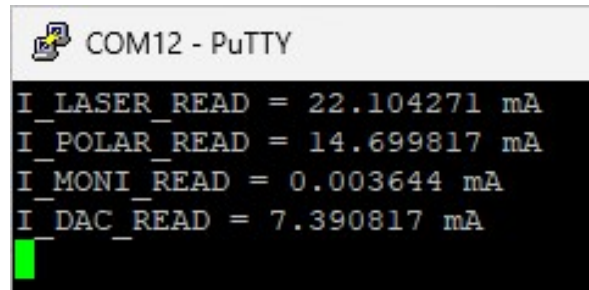
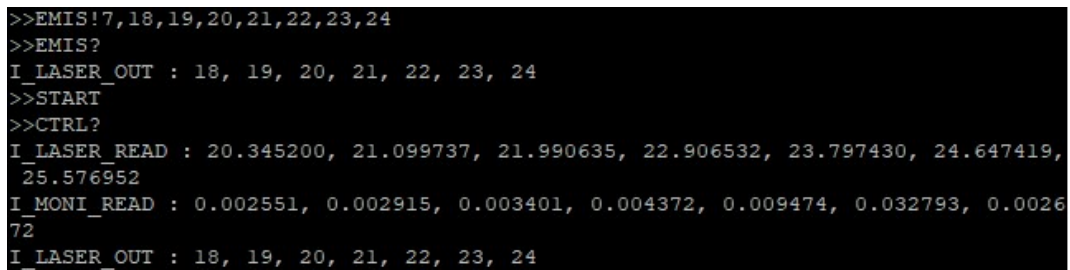


FIGURE 8 – Affichage dans le terminal PuTTY des courants expérimentaux calculés grâce à la lecture de tension sur l’ADC.

2.2 Question 4 : Pilotage du DAC

Nous allons à présent réaliser un programme permettant d’envoyer un nombre (NbSamples) de valeurs stockées dans un tableau sur la sortie du DAC V_{DAC} à une certaine cadence (Period). Pour se faire, nous écrivons directement la commande EMIS! et START. Afin de contrôler la saisie des valeurs, nous réalisons également la commande EMIS?. Enfin, nous codons la commande CTRL? pour vérifier que l’émission a été correcte.

Pour tester notre fonction, nous choisissons de générer une rampe allant de 18 mA à 24 mA, comme le montre la figure 9. Les valeurs données pour générer la rampe sont correctement lues comme le montre EMIS?. Nous pouvons constater un décalage entre la tension voulue pour le laser et la tension réelle mesurée. Ce décalage est a priori normal et dû à la carte. Toutefois, il n’est pas linéaire, ce qui détériore légèrement la rampe. La figure 10, permet de mettre en valeur cette non-linéarité de la différence entre le signal voulu et obtenu. Nous pouvons noter que lorsque l’intensité croît, l’écart diminue.



```
>>EMIS!7,18,19,20,21,22,23,24
>>EMIS?
I_LASER_OUT : 18, 19, 20, 21, 22, 23, 24
>>START
>>CTRL?
I_LASER_READ : 20.345200, 21.099737, 21.990635, 22.906532, 23.797430, 24.647419,
25.576952
I_MONI_READ : 0.002551, 0.002915, 0.003401, 0.004372, 0.009474, 0.032793, 0.0026
72
I_LASER_OUT : 18, 19, 20, 21, 22, 23, 24
```

FIGURE 9 – Génération d’une rampe. Les valeurs sont données par EMIS!. La bonne réception des valeurs et vérifiée grâce à EMIS?. La rampe est générée par START. La vérification de l’émission est donnée par CTRL?.

Note :

Le décalage est bien dû à la carte. Après un échange de carte avec des camarades, nous avons constaté que la différence n’était pas la même, et même inversée (l’intensité réelle inférieure à celle voulue).


```

17  USART0_Write_String("|
    |\n\r");
18  USART0_Write_String("
    -----\n\r");
19  USART0_Write_String("Please enter LIST? to display the available commands.
    Enter LISTD for a more detailed version (in French).\n\r");
20
21  //Appel aux commandes tapees dans la console
22  while(1){
23      USART0_Read_String(Full_Command);
24      for(int i = 0; i < 5; i++){
25          //on recupere la commande
26          Name_Command[i] = Full_Command[i];
27      }
28      Name_Command[6] = "\0";
29      if(!strcmp(Name_Command, "LABV=")){LABV(Full_Command);}
30      if(!strcmp(Name_Command, "*IDN?")){IDN_ASK();}
31      if(!strcmp(Name_Command, "ILAS=")){ILAS(Full_Command);}
32      if(!strcmp(Name_Command, "ILAS?")){ILAS_ASK();}
33      if(!strcmp(Name_Command, "IDAC?")){IDAC_ASK();}
34      if(!strcmp(Name_Command, "IPOL?")){IPOL_ASK();}
35      if(!strcmp(Name_Command, "IMON?")){IMON_ASK();}
36      if(!strcmp(Name_Command, "TIME=")){TIME(Full_Command);}
37      if(!strcmp(Name_Command, "TIME?")){TIME_ASK();}
38      if(!strcmp(Name_Command, "EMIS!")){EMIS(Full_Command);}
39      if(!strcmp(Name_Command, "EMIS?")){EMIS_ASK();}
40      if(!strcmp(Name_Command, "START")){START();}
41      if(!strcmp(Name_Command, "STOP")){STOP();}
42      if(!strcmp(Name_Command, "CTRL?")){CTRL_ASK();}
43      if(!strcmp(Name_Command, "LIST?")){LIST_ASK();}
44      if(!strcmp(Name_Command, "LISTD")){LISTD();}
45  }
46 }

```

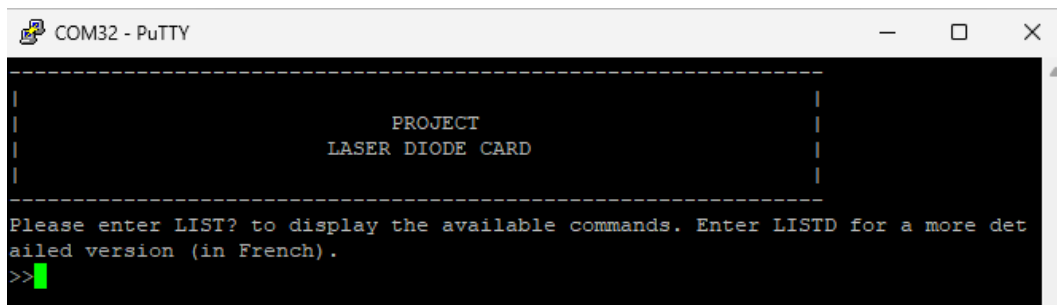


FIGURE 11 – Message au démarrage du programme.

Conclusion

Dans ce projet, nous sommes parvenus à contrôler une diode laser. D'une part, nous pouvons la piloter grâce à des commandes que nous avons définies, mais aussi mesurer les effets de ce pilotage afin de vérifier que celui-ci avait l'effet désiré. Bien que nous pouvons noter un décalage entre les valeurs d'intensité voulues et mesurées du laser, celui-ci est dépendant de l'électronique et ne peut pas être changé.

Enfin, la suite de ce projet est réalisée dans le module d'interfaçage python, dans lequel nous avons développé une interface graphique, utilisant les commandes implémentées dans ce projet.

Annexe

Fichier de commandes : Commands_Driver.c

Dépendances et variables

```
1 //=====//
2 // COURTAY Antoine - ENSSAT //
3 //=====//
4 // File : Command Driver C Functions //
5 // Hardware Environment: Ready for XMega //
6 // Build Environment : AVR Studio 7 + AVRGCC //
7 //=====//
8 #include "Commands_Driver.h"
9 #include "I2C_ADS1115_Driver.h"
10 #include "LED_BP_Buzz_Driver.h"
11
12 //=====//
13 // Variables //
14 //=====//
15 float Vcc = 5;
16 float R5 = 1200;
17 float R6 = 1500;
18 float R7 = 120;
19 float R8 = 220;
20 float R9 = 2700;
21 float R10 = 22000;
22 float Vbe = 0.6;
23
24 float DAC_Value; // Valeur ecrite DAC
25 float DAC_Full = 4096; // Pleine echelle DAC: 2^12
26 float DAC_Values[40]; // Tableau de valeurs ecrites
27
28 unsigned long AD0_Value=0; // Valeur lue ADC channel 0
29 unsigned long AD1_Value=0; // Valeur lue ADC channel 1
30 unsigned long AD2_Value=0; // Valeur lue ADC channel 2
31 unsigned long AD3_Value=0; // Valeur lue ADC channel 3
32
33 // Gain ADC 2/3x soit pleine echelle +/- 6.144V soit 1 bit = 0.1875mV (default
34 // )
35 // 2^16 sur +/- 6.144V
36 // 2^15 sur +6.144V
37 // Soit ADC_Read = 32768 pour 6.144V
38 // Soit ADC_Read = 26667 pour 5V
39 unsigned long ADC_Full = 26667; // Pleine echelle ADC
40
41 float AD0_Values[40]; // Tableau de valeurs lues channel 0
42 float AD1_Values[40]; // Tableau de valeurs lues channel 1
43 float AD2_Values[40]; // Tableau de valeurs lues channel 2
44
45 float I_POLAR_theorique = 18; //(int)(1000*(((R6*Vcc)/(R5+R6))-Vbe)/R7));
46
47 unsigned int I_LASER_voulu = 0;
48
49 float I_MONI_experimental = 0;
50 float I_DAC_experimental = 0;
```



```

50 float I_POLAR_experimental = 0;
51 float I_LASER_experimental = 0; // = I_POLAR_experimental + I_DAC_experimental
    ;
52
53 unsigned int I_LASER_Output_Samples_Nb;
54 unsigned int I_LASER_Output_Samples[40];
55
56 float I_MONI_Input_Samples[40];
57 float I_LASER_Input_Samples[40];
58 float I_DAC_Input_Samples[40];
59 float I_POLAR_Input_Samples[40];
60
61 unsigned int Period = 100;
62
63 char Buffer[100];
64
65 unsigned char Terminal_Labview = 0; // 0: Terminal / 1: Labview

```

Commande *IDN?

```

1 //=====//
2 // Function: IDN_ASK //
3 //=====//
4 // But: Recuperer les numeros serie carte //
5 //=====//
6 void IDN_ASK(void)
7 {
8     if(!Terminal_Labview)
9     {
10         //mode Terminal Putty
11         sprintf(Buffer, "-> IDN: ATXMEGA 128A1\tDevice ID: %d%d%d\tRevision ID: %d\tJTAG USER ID: %d\r\n", MCU.DEVID0 , MCU.DEVID1 , MCU.DEVID2, MCU.REVID, MCU.JTAGUID);
12     }
13     else
14     {
15         //mode Labview
16         sprintf(Buffer, "ATXMEGA 128A1\tDevice ID: %d%d%d\tRevision ID: %d\tJTAG USER ID: %d\r\n", MCU.DEVID0 , MCU.DEVID1 , MCU.DEVID2, MCU.REVID, MCU.JTAGUID);
17     }
18     USART0_Write_String(Buffer);
19 }

```

Commande LABV=

```

1 //=====//
2 // Function: LABV //
3 //=====//
4 // But: Choisir Pilotage Terminal ou Labview //
5 //=====//
6 void LABV(char Full_Command[100])
7 {
8     sscanf (Full_Command, "LABV=%d%s", &Terminal_Labview, Full_Command);

```

9 }

Voici ton texte avec tous les accents remplacés par des lettres non accentuées :
“latex

Commande ILAS=

```
1 //=====//
2 // Fonction ILAS
3 //=====//
4 // But: Fixer 18mA <= nouveau courant <= 35mA et envoi DAC
5 //=====//
6 void ILAS(char Full_Command[100])
7 {
8     //on retire "ILAS="
9     sscanf (Full_Command, "ILAS=%d%s", &I_LASER_voulu, Full_Command);
10    //on definit le message d'erreur
11    char pERROR_ILAS[50] = "ILAS doit etre compris entre 18 et 35 mA.\n\r";
12
13    //on teste si ILAS est comprise dans les valeurs acceptables
14    if(I_LASER_voulu < 18 || I_LASER_voulu > 35){
15        ERROR(pERROR_ILAS);
16    }
17    else{
18        AD2_Value = ADS1115_Read(2);
19        I_POLAR_experimental = (((float)AD2_Value * 5 / ADC_Full) / R7)*1000;
20        DAC_Value = ((I_LASER_voulu - I_POLAR_experimental) * 0.001 * R8 + Vbe)*
        DAC_Full/5;
21        char *pDAC = NULL;
22
23        MCP4725_Write((int)(DAC_Value));
24    }
25 }
```

Commande ILAS ?

```
1 //=====//
2 // Fonction ILAS_ASK
3 //=====//
4 // But: Afficher la valeur du courant I_LASER_READ
5 //=====//
6 void ILAS_ASK(void)
7 {
8     char pI_LASER_experimental[100];
9     char pIa[100];
10    char pIb[100];
11
12    //on recupere les valeurs de tension mesurees par l'ADC
13    AD1_Value = ADS1115_Read(1);
14    AD2_Value = ADS1115_Read(2);
15
16    char *pV_AD1 = NULL;
17    char *pV_AD2 = NULL;
18    sprintf(pV_AD1, "V_AD1 = %lu\n\r", AD1_Value);
```

```

19  USART0_Write_String(pV_AD1);
20  sprintf(pV_AD2, "V_AD2 = %lu\n\r", AD2_Value);
21  USART0_Write_String(pV_AD2);
22
23  //on calcule I_LASER_READ
24  I_POLAR_experimental = (((float)AD2_Value * 5) / ADC_Full) / R7;
25  I_DAC_experimental = (((float)AD1_Value * 5) / ADC_Full) / R8;
26  I_LASER_experimental = (I_POLAR_experimental + I_DAC_experimental)*1000;
27
28  if(Terminal_Labview){
29      //mode Labview
30      sprintf(pI_LASER_experimental, "%f\n\r", I_LASER_experimental);
31  }
32  else{
33      //mode Terminal Putty
34      sprintf(pIa, "I_POLAR_READ = %f mA\n\r", I_POLAR_experimental);
35      sprintf(pIb, "I_DAC_READ = %f mA\n\r", I_DAC_experimental);
36      sprintf(pI_LASER_experimental, "I_LASER_READ = %f mA\n\r",
37          I_LASER_experimental);
38  }
39  USART0_Write_String(pIa);
40  USART0_Write_String(pIb);
41  USART0_Write_String(pI_LASER_experimental);
42 }

```

Commande IMON ?

```

1  //=====//
2  // Fonction IMON_ASK
3  //=====//
4  // But: Afficher la valeur du courant I_MONI_READ
5  //=====//
6  void IMON_ASK(void)
7  {
8      char pI_MONI_experimental[50];
9
10     //on recupere les valeurs de tension mesurees par l'ADC
11     AD0_Value = ADS1115_Read(0);
12
13     //on calcule I_MONI_READ
14     I_MONI_experimental = (((float)AD0_Value * 5 / ADC_Full) / (R10 + R9))*1000;
15
16     if(Terminal_Labview){
17         //mode Labview
18         sprintf(pI_MONI_experimental, "%f\n\r", I_MONI_experimental);
19     }
20     else{
21         //mode Terminal Putty
22         sprintf(pI_MONI_experimental, "I_MONI_READ = %f mA\n\r",
23             I_MONI_experimental);
24     }
25     USART0_Write_String(pI_MONI_experimental);
26 }

```

Commande IPOL ?

```
1 //=====//
2 // Fonction IPOL_ASK
3 //=====//
4 // But: Afficher la valeur du courant I_POLAR_READ
5 //=====//
6 void IPOL_ASK(void)
7 {
8     char pI_POLAR_experimental[50];
9
10    //on recupere les valeurs de tension mesurees par l'ADC
11    AD2_Value = ADS1115_Read(2);
12
13    //on calcule I_POLAR_READ
14    I_POLAR_experimental = (((float)AD2_Value * 5 / ADC_Full) / R7)*1000;
15
16    if(Terminal_Labview){
17        //mode Labview
18        sprintf(pI_POLAR_experimental, "%f\n\r", I_POLAR_experimental);
19    }
20    else{
21        //mode Terminal Putty
22        sprintf(pI_POLAR_experimental, "I_POLAR_READ = %f mA\n\r",
23            I_POLAR_experimental);
24    }
25    USART0_Write_String(pI_POLAR_experimental);
26 }
```

Commande IDAC ?

```
1 //=====//
2 // Fonction IDAC_ASK
3 //=====//
4 // But: Afficher la valeur du courant I_DAC_READ
5 //=====//
6 void IDAC_ASK(void)
7 {
8     char pI_DAC_experimental[50];
9
10    //on recupere les valeurs de tension mesurees par l'ADC
11    AD1_Value = ADS1115_Read(1);
12
13    //on calcule I_DAC_READ
14    I_DAC_experimental = (((float) AD1_Value * 5 / ADC_Full) / R8)*1000;
15
16    if(Terminal_Labview){
17        //mode Labview
18        sprintf(pI_DAC_experimental, "%f\n\r", I_DAC_experimental);
19    }
20    else{
21        //mode Terminal Putty
22        sprintf(pI_DAC_experimental, "I_DAC_READ = %f mA\n\r", I_DAC_experimental)
23        ;
24    }
25 }
```

```

24     USART0_Write_String(pI_DAC_experimental);
25 }

```

Commande EMIS !

```

1  //=====//
2  // Fonction EMIS
3  //=====//
4  // But: Construire le tableau I_LASER_OUT
5  //=====//
6  void EMIS(char Full_Command[100])
7  {
8      //on retire "EMIS!"
9      sscanf (Full_Command, "EMIS!%d%s", &I_LASER_Output_Samples_Nb, Full_Command)
10         ;
11     uint16_t size_Full_Command = strlen(Full_Command)/3;
12     if(size_Full_Command != I_LASER_Output_Samples_Nb){
13         //sequence d'erreur : le nombre d'echantillons est different de celui
14         //annonce
15         char pERROR_EMIS[50] = "Il n'y a pas le bon nombre de valeurs indiquees.\n
16         \r";
17         ERROR(pERROR_EMIS);
18     }
19     for(int j = 0; j < I_LASER_Output_Samples_Nb; j++){
20         sscanf(Full_Command, "%d%s", &I_LASER_Output_Samples[j], Full_Command);
21         if(I_LASER_Output_Samples[j] < 18 || I_LASER_Output_Samples[j] > 35){
22             //sequence d'erreur : la valeur de I_LASER_OUT n'est pas dans les bornes
23             //acceptables
24             char pERROR_ILAS[50] = "ILAS doit etre compris entre 18 et 35 mA.\n\r";
25             ERROR(pERROR_ILAS);
26         }
27     }
28 }

```

Fonction d'erreur

```

1  //=====//
2  // Fonction ERROR
3  //=====//
4  // But: Activer la sequence d'erreur et afficher le message d'erreur
5  //=====//
6  void ERROR(char pERROR[50]){
7      //affichage du message d'erreur
8      USART0_Write_String(pERROR);
9
10     //initialisation
11     BUZZ_Init();
12     LED_Init();
13     uint8_t State = 0;
14
15     //demarrage de la sequence d'erreur
16     for (int i = 0; i<100; i++ ){
17         BUZZ_Go(440);

```

```

18     switch(State){
19         case 0 :
20             LED_Off(1);
21             LED_Off(2);
22             State ++;
23             break;
24         case 1 :
25             LED_On(1);
26             LED_Off(2);
27             State ++;
28             break;
29         case 2 :
30             LED_On(2);
31             LED_Off(1);
32             State = 0;
33             break;
34         default :
35             LED_Off(1);
36             LED_Off(2);
37             State = 0;
38             break;
39     }
40 }
41 LED_Off(1);
42 LED_Off(2);
43 }

```

Commande EMIS ?

```

1  //=====//
2  // Fonction EMIS_ASK
3  //=====//
4  // But: Afficher le tableau I_LASER_OUT
5  //=====//
6  void EMIS_ASK(void)
7  {
8      if(!Terminal_Labview){
9          //mode Terminal Putty
10         USART0_Write_String("I_LASER_OUT :");
11     }
12
13     //on affiche la premiere valeur
14     char *pI_LASER_Output_Samples = NULL;
15     sprintf(pI_LASER_Output_Samples, " %u", I_LASER_Output_Samples[0]);
16     USART0_Write_String(pI_LASER_Output_Samples);
17
18     //on affiche les autres valeurs s'il y en a
19     for(int j = 1; j < I_LASER_Output_Samples_Nb; j++){
20         char *pI_LASER_Output_Samples = NULL;
21         sprintf(pI_LASER_Output_Samples, " %u", I_LASER_Output_Samples[j]);
22         USART0_Write_String(pI_LASER_Output_Samples);
23     }
24     USART0_Write_String("\n\r");
25 }

```

Commande TIME=

```
1 //=====//
2 // Fonction TIME
3 //=====//
4 // But: Regler la periode d'emission des echantillons du tableau I_LASER_OUT
5 //=====//
6 void TIME(char Full_Command[100])
7 {
8     unsigned int Period_temp = 0;
9
10    //on retire "TIME="
11    sscanf (Full_Command, "TIME=%u", &Period_temp, Full_Command);
12
13    if(Period_temp < 10 || Period_temp > 1000){
14        //sequence d'erreur : la periode n'est pas comprise dans les valeurs
15        //acceptables
16        char pERROR_TIME[50] = "TIME doit etre compris entre 10 et 1000 ms.\n\r"
17        ;
18        ERROR(pERROR_TIME);
19    }
20    else{
21        //mise a jour de la periode
22        Period = Period_temp;
23    }
24 }
```

Commande TIME ?

```
1 //=====//
2 // Fonction TIME_ASK
3 //=====//
4 // But: Afficher la valeur de la periode d'emission des echantillons
5 //=====//
6 void TIME_ASK(void)
7 {
8     char pPeriod[50];
9
10    if(Terminal_Labview){
11        //mode Labview
12        sprintf(pPeriod, "%u\n\r", Period);
13    }
14    else{
15        //mode Terminal Putty
16        sprintf(pPeriod, "Period = %u ms\n\r", Period);
17    }
18    USART0_Write_String(pPeriod);
19 }
```

Commande START

```
1 //=====//
2 // Fonction START
3 //=====//
```

```

4 // But: Emmettre le contenu du tableau I_LASER_OUT a la bonne periode et
  mesurer
5 //      dans un tableau les valeurs des courants I_LASER_READ et I_MONI_READ
6 //=====//
7 void START(void)
8 {
9     for(int i = 0; i < I_LASER_Output_Samples_Nb; i++){
10         //on retire "ILAS="
11         char *ilasout = NULL;
12         sprintf(ilasout, "ILAS=%d", I_LASER_Output_Samples[i]);
13         //on met a jour la valeur de tension d'alimentation du laser
14         ILAS(ilasout);
15
16         //on recupere les valeurs de tension mesurees par l'ADC
17         ADO_Value = ADS1115_Read(0);
18         //on calcule I_MONI_READ
19         I_MONI_experimental = (((float)ADO_Value * 5 / ADC_Full) / (R10 + R9))
*1000;
20         //on ajoute la valeur de I_MONI_READ au tableau
21         I_MONI_Input_Samples[i] = I_MONI_experimental;
22
23         //on recupere les valeurs de tension mesurees par l'ADC
24         AD1_Value = ADS1115_Read(1);
25         AD2_Value = ADS1115_Read(2);
26         //on calcule I_LASER_READ
27         I_POLAR_experimental = (((float)AD2_Value * 5) / ADC_Full) / R7;
28         I_DAC_experimental = (((float)AD1_Value * 5) / ADC_Full) / R8;
29         I_LASER_experimental = (I_POLAR_experimental + I_DAC_experimental)
*1000;
30         //on ajoute la valeur de I_LASER_READ au tableau
31         I_LASER_Input_Samples[i] = I_LASER_experimental;
32
33         //on attend pour avoir la bonne periode d'emission
34         for(unsigned int j = 0; j < Period; j++){
35             _delay_ms(1);
36         }
37     }
38 }

```

Commande STOP

```

1 //=====//
2 // Fonction STOP
3 //=====//
4 // But: Eteindre le DAC
5 //=====//
6 void STOP(void)
7 {
8     MCP4725_Write(0);
9 }

```

Commande CTRL ?

```

1 //=====//

```



```

2 // Fonction CTRL_ASK
3 //=====//
4 // But: Afficher les tableaux des courants I_LASER_READ, I_MONI_READ et
5 //      I_LASER_OUT
6 //=====//
7 void CTRL_ASK(void)
8 {
9     if(!Terminal_Labview){
10         //mode Terminal Putty
11         USART0_Write_String("I_LASER_READ :");
12     }
13     //on affiche la premiere valeur de I_LASER_READ
14     char *pI_LASER_Input_Samples = NULL;
15     sprintf(pI_LASER_Input_Samples, " %f", I_LASER_Input_Samples[0]);
16     USART0_Write_String(pI_LASER_Input_Samples);
17     //on affiche les autres valeurs
18     for(int j = 1; j < I_LASER_Output_Samples_Nb; j++){
19         char *pI_LASER_Input_Samples = NULL;
20         sprintf(pI_LASER_Input_Samples, " , %f", I_LASER_Input_Samples[j]);
21         USART0_Write_String(pI_LASER_Input_Samples);
22     }
23     USART0_Write_String("\n\r");
24
25     if(!Terminal_Labview){
26         //mode Terminal Putty
27         USART0_Write_String("I_MONI_READ :");
28     }
29     //on affiche la premiere valeur de I_MONI_READ
30     char *pI_MONI_Input_Samples = NULL;
31     sprintf(pI_MONI_Input_Samples, " %f", I_MONI_Input_Samples[0]);
32     USART0_Write_String(pI_MONI_Input_Samples);
33     //on affiche les autres valeurs
34     for(int j = 1; j < I_LASER_Output_Samples_Nb; j++){
35         char *pI_MONI_Input_Samples = NULL;
36         sprintf(pI_MONI_Input_Samples, " , %f", I_MONI_Input_Samples[j]);
37         USART0_Write_String(pI_MONI_Input_Samples);
38     }
39     USART0_Write_String("\n\r");
40
41     //on affiche les avleurs de I_LASER_OUT
42     EMIS_ASK();
43 }

```

Commande LIST ?

```

1 //=====//
2 // Fonction LIST?
3 //=====//
4 // But: Afficher la liste des commandes dans le terminal (en anglais)
5 //=====//
6 void LIST_ASK(void)
7 {
8     USART0_Write_String("- LABV=x\tApplication driven by Terminal Putty (x=0)
9     or LabView (x=1)\n\r");

```

```

9     USART0_Write_String("- *IDN?\tDisplay serial numbers of the
microcontroller and its programmer\n\r");
10    USART0_Write_String("- ILAS=x\tChoose I_LASER_WRITE (18mA <= I_LASER_WRITE
<= 35mA)\n\r");
11    USART0_Write_String("- ILAS?\tDisplay I_LASER_READ measured\n\r");
12    USART0_Write_String("- IDAC?\tDisplay I_DAC_READ measured\n\r");
13    USART0_Write_String("- IPOL?\tDisplay I_POLAR_READ measured\n\r");
14    USART0_Write_String("- IMON?\tDisplay I_MONI_READ measured\n\r");
15    USART0_Write_String("- TIME=x\tChoose the emission period (10ms <= Period
<= 1000ms)\n\r");
16    USART0_Write_String("- TIME?\tDisplay the period value\n\r");
17    USART0_Write_String("- EMIS!x,a,b,c,... \tBuild the table of measures to
be issued with x=NbSamples et a,b,c,... the I_LASER_WRITE values in mA\n\r
");
18    USART0_Write_String("- EMIS?\tDisplay the table of current values
constructed with the EMIS! command\n\r");
19    USART0_Write_String("- START\tDrive the LED with the samples indicated
with EMIS!x,a,b,c... and Period, and measure the values of I_LASER_READ /
I_MONI_READ for every sample\n\r");
20    USART0_Write_String("- STOP\tTurn off the DAC\n\r");
21    USART0_Write_String("- CTRL?\tDisplay the measured values of the current
I_LASER_READ, I_MONI_READ and I_LASER_WRITE following the command START\n\r
");
22    USART0_Write_String("- LIST?\tDisplay this command list in the terminal\n\r
");
23    USART0_Write_String("- LISTD\tDisplay a more detailed command list of
these commands in French in the terminal\n\r");
24 }

```

Commande LISTD

```

1 //=====//
2 // Fonction LISTD
3 //=====//
4 // But: Afficher la liste detaillees des commandes dans le terminal (en
francais)
5 //=====//
6 void LISTD(void)
7 {
8     USART0_Write_String("- LABV=x\n\rApplication pilotee par Terminal Putty (x
=0) ou LabView (x=1). En fonction de x, certains affichages envoyes sur la
liaison UART peuvent etre caches.\n\r");
9     USART0_Write_String("- *IDN?\n\rAffiche les numeros de serie du
microcontrolleur et de son programmeur\n\r");
10    USART0_Write_String("- ILAS=x \n\rPositionne I_LASER_WRITE avec 18mA <=
I_LASER_WRITE <= 35mA (Ecriture V_DAC)\n\r");
11    USART0_Write_String("- ILAS?\n\rAffiche I_LASER_READ mesure (Lecture V_AD1
, V_AD2 et mise en forme equation partie etude)\n\r");
12    USART0_Write_String("- IDAC?\n\rAffiche I_DAC_READ mesure (Lecture V_AD1
et mise en forme equation partie etude)\n\r");
13    USART0_Write_String("- IPOL?\n\rAffiche I_POLAR_READ mesure (Lecture V_AD2
et mise en forme equation partie etude)\n\r");
14    USART0_Write_String("- IMON?\n\rAffiche I_MONI_READ mesure (Lecture V_ADO
et mise en forme equation partie etude)\n\r");
15    USART0_Write_String("- TIME=x\n\rPositionne la periode d'emission (10ms <=

```

```

    Period <= 1000ms) des echantillons du tableau I_LASER_OUT[NbSamples] (
    tableau construit par la commande EMIS!)\n\r");
16  USARTO_Write_String("- TIME?\n\rAffiche la valeur courante du parametre
    Period\n\r");
17  USARTO_Write_String("- EMIS!x,a,b,c,... \n\rConstruit le tableau
    I_LASER_OUT[NbSamples] a emettre avec x=NbSamples et a,b,c,... les
    echantillons I_LASER_WRITE en mA\n\r");
18  USARTO_Write_String("- EMIS?\n\rAffiche le tableau des valeurs de courant
    I_LASER_OUT[NbSamples] construit avec la commande EMIS!\n\r");
19  USARTO_Write_String("- START\n\rPilote la LED avec les echantillons du
    tableau I_LASER_OUT[NbSamples] a chaque instant Period et mesure dans un
    tableau les courants I_LASER_READ[NbSamples] /I_MONI_READ[NbSamples] pour
    chaque echantillon\n\r");
20  USARTO_Write_String("- STOP\n\rEteint le DAC\n\r");
21  USARTO_Write_String("- CTRL?\n\rAffiche les tableaux des courants
    I_LASER_READ[NbSamples]/I_MONI_READ[NbSamples] mesures (et I_LASER_OUT[
    NbSamples] pour comparaison) suite a la commande START\n\r");
22  USARTO_Write_String("- LIST?\n\rAffiche une version synthetique de cette
    liste de commandes dans le terminal pour l'utilisateur\n\r");
23  USARTO_Write_String("- LISTD\n\rAffiche cette liste de commandes dans le
    terminal pour l'utilisateur\n\r");
24 }

```